

LAB SESSION # 01

```
In [2]: passed = 0
failed = 0
for i in range(1, 11):
    # Read tensile strength for each rod
    strength = float(input(f"Enter tensile strength for rod {i} (in MPa): "))    #i
    if strength >= 250:
        print(f"Rod {i}: Pass")
        passed += 1
    else:
        print(f"Rod {i}: Fail")
        failed += 1
print("\n--- Test Summary ---")
print(f"Total Passed Rods: {passed}")
print(f"Total Failed Rods: {failed}")
```

Rod 1: Fail
Rod 2: Fail
Rod 3: Pass
Rod 4: Pass
Rod 5: Pass
Rod 6: Pass
Rod 7: Pass
Rod 8: Pass
Rod 9: Fail
Rod 10: Pass

--- Test Summary ---
Total Passed Rods: 7
Total Failed Rods: 3

LAB SESSION # 02

```
In [4]: within_tolerance = 0
out_of_tolerance = 0
for i in range(1, 6):

    diameter = float(input(f"Enter diameter of shaft {i} (in mm): "))

    if 49 <= diameter <= 51:
        print(f"Shaft {i}: Within Tolerance")
        within_tolerance += 1    # for checking range
    else:
        print(f"Shaft {i}: Out of Tolerance")
        out_of_tolerance += 1

print("\n--- Inspection Summary ---")
print(f"Total Shafts Within Tolerance: {within_tolerance}")
print(f"Total Shafts Out of Tolerance: {out_of_tolerance}")
```

Shaft 1: Out of Tolerance
Shaft 2: Out of Tolerance
Shaft 3: Out of Tolerance
Shaft 4: Within Tolerance
Shaft 5: Within Tolerance

--- Inspection Summary ---
Total Shafts Within Tolerance: 2
Total Shafts Out of Tolerance: 3

LAB SESSION # 03

```
In [5]: def check_hardness(value):  
        if 150 <= value <= 200:  
            print("Pass")  
            return True  
        else:  
            print("Fail")  
            return False  
  
        print("Testing hardness for 8 components (1 to 8):")  
  
        passed = 0  
        failed = 0  
  
        for i in range(1, 9):  
            hardness = float(input(f"Enter hardness value for component {i} (in HB): "))  
  
            if check_hardness(hardness):  
                passed += 1  
            else:  
                failed += 1  
        print("\n--- Hardness Test Summary ---") #/n means print in next line  
        print(f"Total Components Tested: 8")  
        print(f"Passed Components: {passed}")  
        print(f"Failed Components: {failed}")
```

Testing hardness for 8 components (1 to 8):

Fail
Fail
Fail
Fail
Fail
Fail
Fail
Fail
Fail

--- Hardness Test Summary ---
Total Components Tested: 8
Passed Components: 0
Failed Components: 8

LAB SESSION # 04

In [6]: `import numpy as np`

```
data = np.array(list(map(float, input("Enter 12 diameters (mm) separated by spaces:
tolerance_mask = (data >= 49) & (data <= 51)
labels = np.where(tolerance_mask, "Within Tolerance", "Out of Tolerance")
total_shafts = data.size
within_tolerance = np.sum(tolerance_mask)
out_of_tolerance = total_shafts - within_tolerance
out_of_tolerance_indices = np.where(~tolerance_mask)[0]
print("\n=== Shaft Inspection Report ===")
print("Total shafts tested:", total_shafts)
print("Number within tolerance:", within_tolerance)
print("Number out of tolerance:", out_of_tolerance)
print("Indices of out-of-tolerance shafts:", out_of_tolerance_indices)

print("\n=== Statistical Summary ===")
print("Mean diameter:", np.mean(data))
print("Standard deviation (sample):", np.std(data, ddof=1))
print("Minimum diameter:", np.min(data))
print("Maximum diameter:", np.max(data))

print("\nLabels for each shaft:")
print(labels)
```

=== Shaft Inspection Report ===

Total shafts tested: 1

Number within tolerance: 0

Number out of tolerance: 1

Indices of out-of-tolerance shafts: [0]

=== Statistical Summary ===

Mean diameter: 48.0

Standard deviation (sample): nan

Minimum diameter: 48.0

Maximum diameter: 48.0

Labels for each shaft:

['Out of Tolerance']

```
c:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\_core
_methods.py:222: RuntimeWarning: Degrees of freedom <= 0 for slice
    ret = _var(a, axis=axis, dtype=dtype, out=out, ddof=ddof,
c:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\_core
_methods.py:214: RuntimeWarning: invalid value encountered in scalar divide
    ret = ret.dtype.type(ret / rcount)
```

LAB SESSION # 05

In [7]: `import seaborn as sns
import pandas as pd`

```
# Load dataset
df = sns.load_dataset("tips")

# rows before cleaning
before_rows = len(df)
```

```
# display data
print(df.head())
print(df.info())

# remove missing values
df = df.dropna()

# create new column
df["tip_percentage"] = (df["tip"] / df["total_bill"]) * 100

# filter data
df = df[df["total_bill"] > 20]

# save to excel
df.to_excel("tips_cleaned.xlsx", index=False)

# read excel file again
new_df = pd.read_excel("tips_cleaned.xlsx")

print(new_df.head())

# rows after cleaning
after_rows = len(df)

print("Rows before cleaning:", before_rows)
print("Rows after cleaning:", after_rows)
```

```

      total_bill  tip    sex smoker  day    time  size
0         16.99  1.01  Female    No  Sun  Dinner    2
1         10.34  1.66    Male    No  Sun  Dinner    3
2         21.01  3.50    Male    No  Sun  Dinner    3
3         23.68  3.31    Male    No  Sun  Dinner    2
4         24.59  3.61  Female    No  Sun  Dinner    4
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null     float64
1   tip         244 non-null     float64
2   sex         244 non-null     category
3   smoker      244 non-null     category
4   day         244 non-null     category
5   time        244 non-null     category
6   size        244 non-null     int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
None
      total_bill  tip    sex smoker  day    time  size  tip_percentage
0         21.01  3.50    Male    No  Sun  Dinner    3         16.658734
1         23.68  3.31    Male    No  Sun  Dinner    2         13.978041
2         24.59  3.61  Female    No  Sun  Dinner    4         14.680765
3         25.29  4.71    Male    No  Sun  Dinner    4         18.623962
4         26.88  3.12    Male    No  Sun  Dinner    4         11.607143
Rows before cleaning: 244
Rows after cleaning: 97

```

LAB SESSION 06

```

In [8]: import seaborn as sns
import matplotlib.pyplot as plt

# Load datasets
iris = sns.load_dataset("iris")
tips = sns.load_dataset("tips")

# ----- Line Chart -----
plt.figure()
plt.plot(iris.index, iris["sepal_length"])
plt.title("Line Chart of Sepal Length")
plt.xlabel("Index")
plt.ylabel("Sepal Length")
plt.savefig("line_chart.png")
plt.show()

# ----- Scatter Plot -----
plt.figure()
sns.scatterplot(x="sepal_length", y="petal_length", data=iris, hue="species")
plt.title("Scatter Plot of Iris Data")
plt.savefig("scatter_plot.png")
plt.show()

# ----- Boxplot -----

```

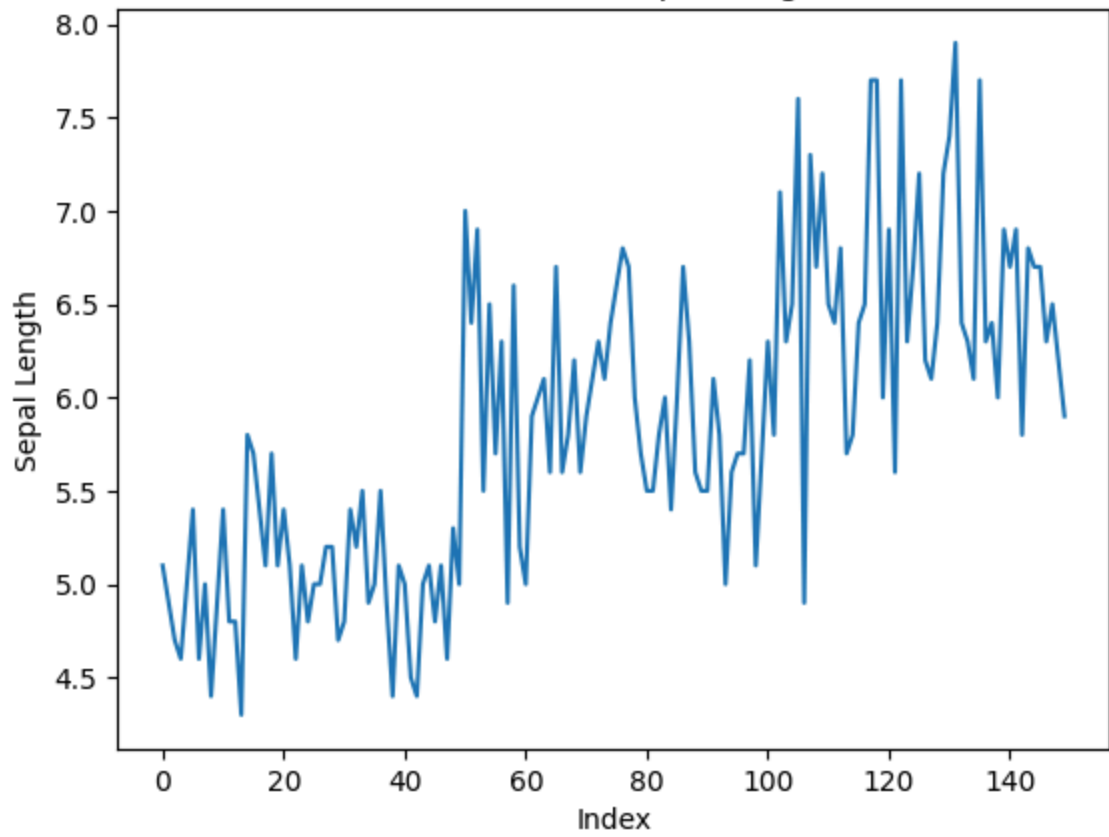
```
plt.figure()
sns.boxplot(x="day", y="total_bill", data=tips)
plt.title("Boxplot of Total Bill by Day")
plt.savefig("boxplot.png")
plt.show()

# ----- Histogram -----
plt.figure()
plt.hist(tips["total_bill"], bins=10)
plt.title("Histogram of Total Bill")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.savefig("histogram.png")
plt.show()

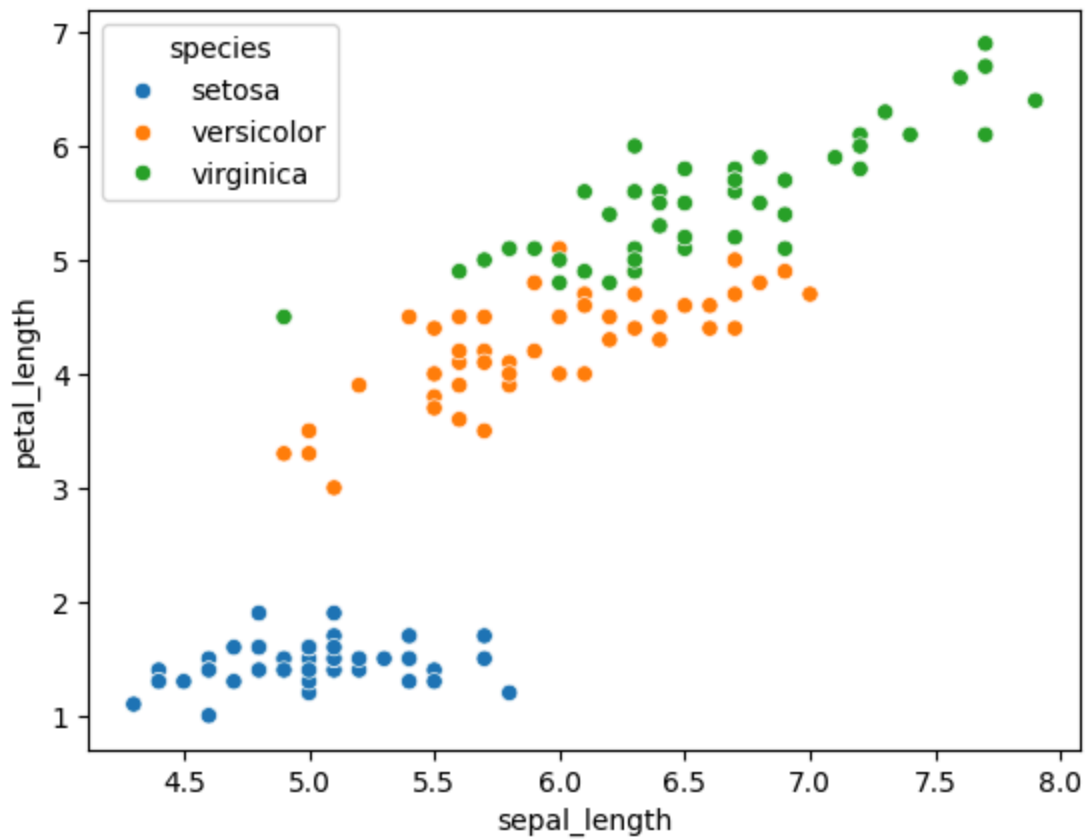
# ----- Pie Chart -----
plt.figure()
tips["day"].value_counts().plot.pie(autopct="%1.1f%%")
plt.title("Pie Chart of Days")
plt.ylabel("")
plt.savefig("pie_chart.png")
plt.show()

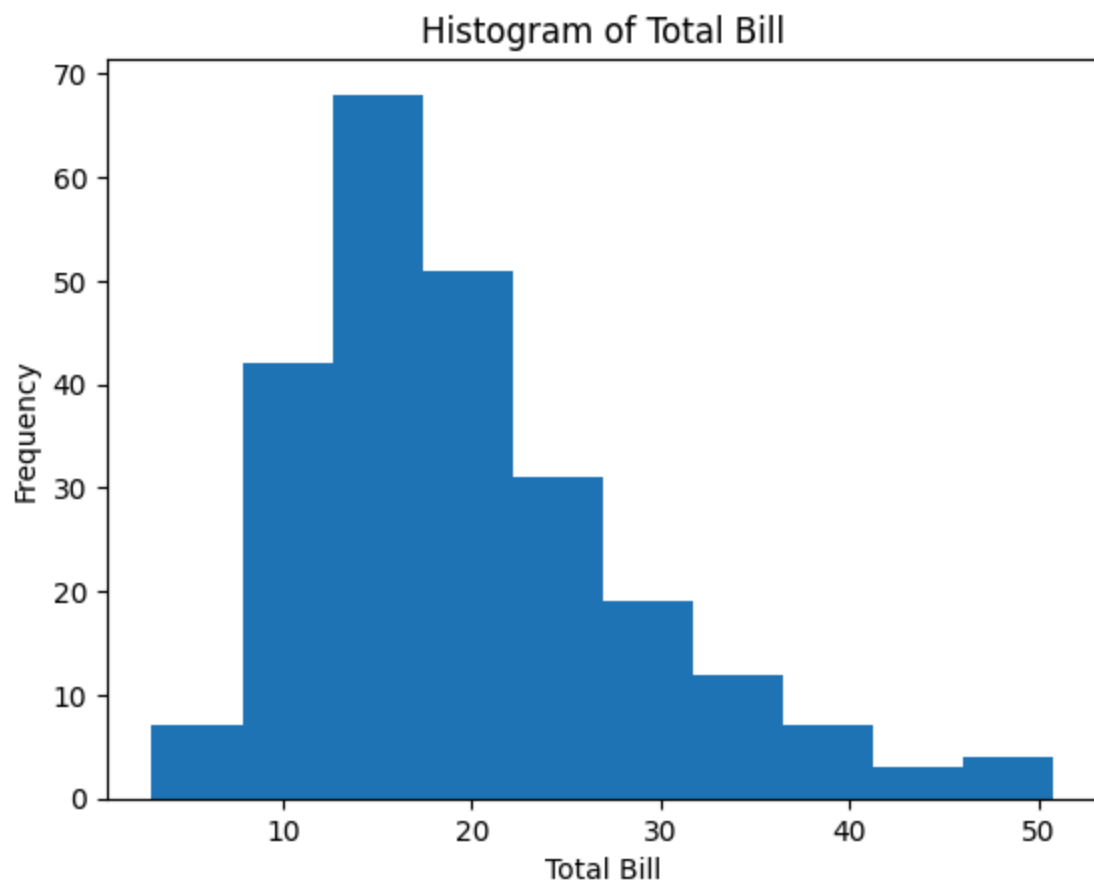
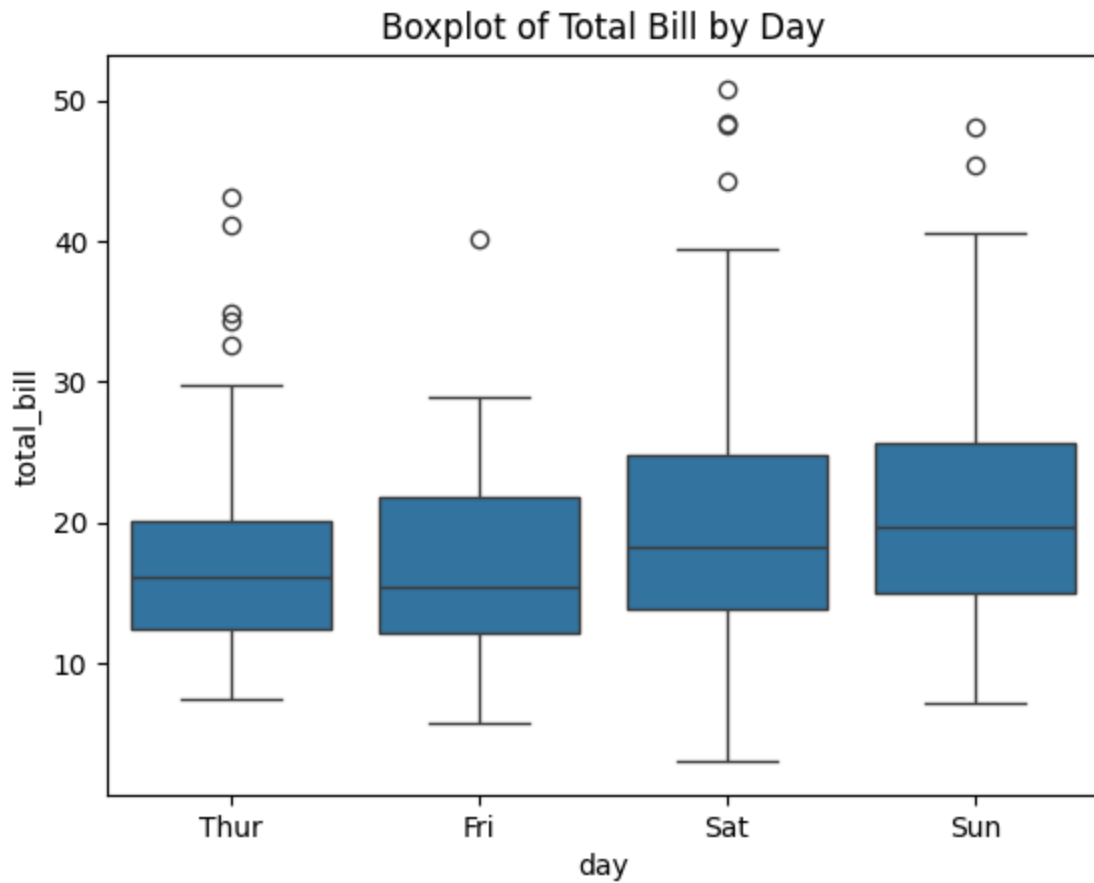
# ----- Bar Chart -----
plt.figure()
sns.barplot(x="day", y="total_bill", data=tips)
plt.title("Average Total Bill by Day")
plt.savefig("bar_chart.png")
plt.show()
```

Line Chart of Sepal Length

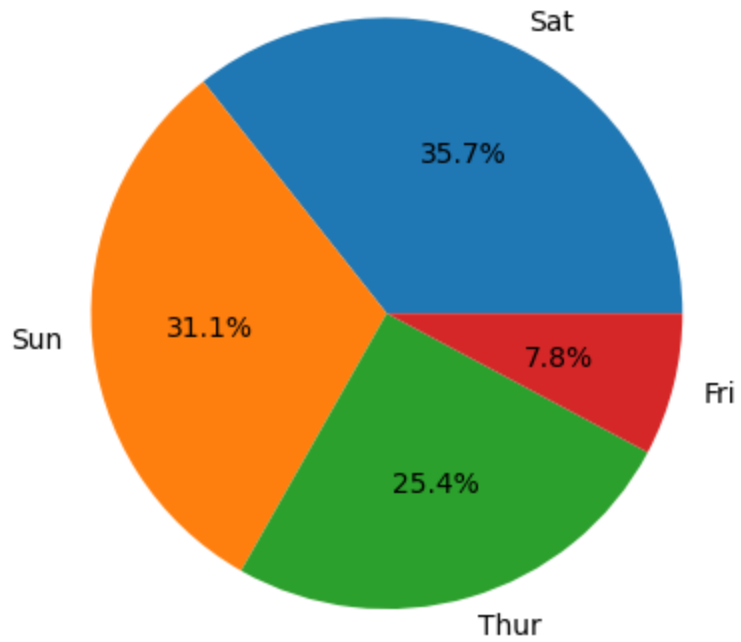


Scatter Plot of Iris Data

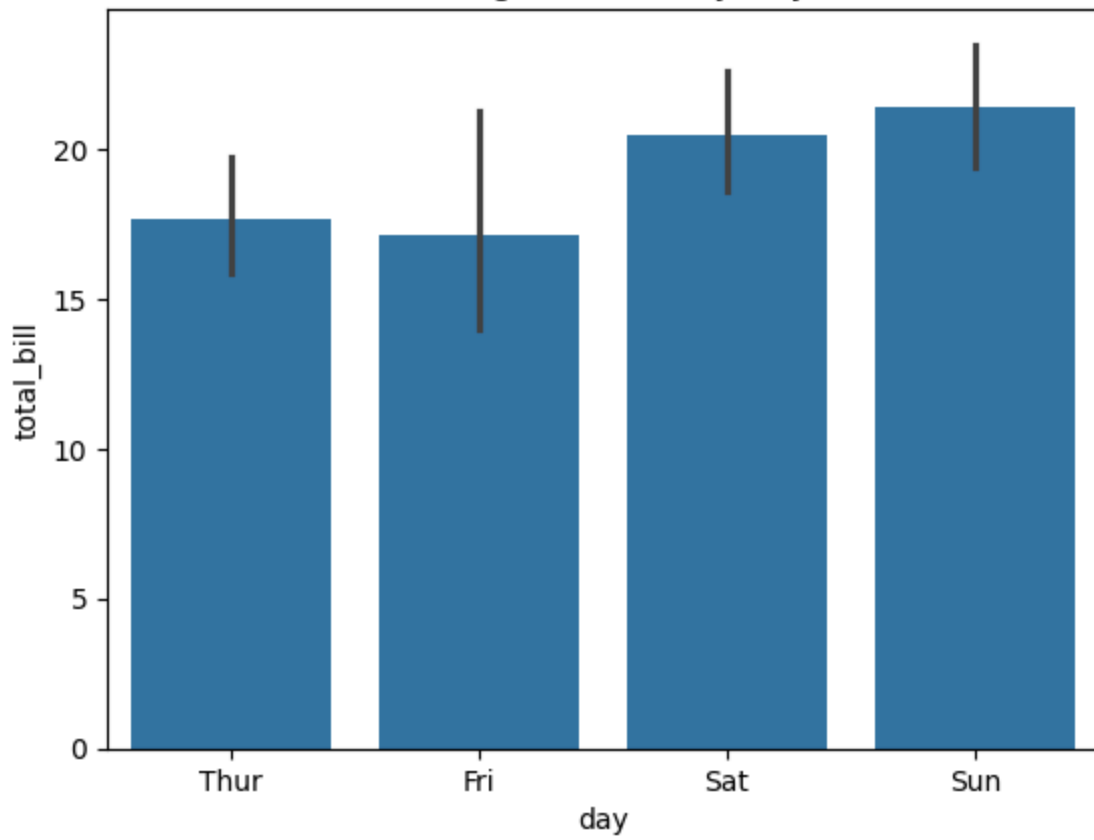




Pie Chart of Days



Average Total Bill by Day



```
In [9]: import pandas as pd
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

# create dataset
data = {
    "Gender": ["Male", "Female", "Male", "Female"],
    "City": ["Lahore", "Karachi", "Islamabad", "Lahore"],
    "Education": ["Matric", "Intermediate", "Bachelor", "Master"]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# ----- Label Encoding (Gender) -----
le = LabelEncoder()
df["Gender_Encoded"] = le.fit_transform(df["Gender"])

# ----- One-Hot Encoding (City) -----
city_encoded = pd.get_dummies(df["City"], prefix="City")
df = pd.concat([df, city_encoded], axis=1)

# ----- Ordinal Encoding (Education) -----
education_order = ["Matric", "Intermediate", "Bachelor", "Master"]
oe = OrdinalEncoder(categories=education_order)
df["Education_Encoded"] = oe.fit_transform(df[["Education"]])

print("\nEncoded Dataset:")
print(df)
```

Original Dataset:

	Gender	City	Education
0	Male	Lahore	Matric
1	Female	Karachi	Intermediate
2	Male	Islamabad	Bachelor
3	Female	Lahore	Master

Encoded Dataset:

	Gender	City	Education	Gender_Encoded	City_Islamabad \
0	Male	Lahore	Matric	1	False
1	Female	Karachi	Intermediate	0	False
2	Male	Islamabad	Bachelor	1	True
3	Female	Lahore	Master	0	False

	City_Karachi	City_Lahore	Education_Encoded
0	False	True	0.0
1	True	False	1.0
2	False	False	2.0
3	False	True	3.0

LAB SESSION # 08

```
In [10]: import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
```

```

# ----- Step 1: Create sample dataset -----
data = {
    "Age": [25, 30, 35, 40, 45],
    "Income": [50000, 60000, 70000, 80000, 90000],
    "Monthly_Spending": [2000, 2500, 3000, 3500, 4000]
}

df = pd.DataFrame(data)
print("Original Dataset:")
print(df)

# ----- Step 2: Min-Max Scaling -----
min_max_scaler = MinMaxScaler()
df_minmax = pd.DataFrame(min_max_scaler.fit_transform(df), columns=df.columns)
print("\nMin-Max Scaled Dataset:")
print(df_minmax)

# ----- Step 3: Standardization (Z-score) -----
std_scaler = StandardScaler()
df_standard = pd.DataFrame(std_scaler.fit_transform(df), columns=df.columns)
print("\nStandardized Dataset (Z-score):")
print(df_standard)

# ----- Step 4: Robust Scaling -----
robust_scaler = RobustScaler()
df_robust = pd.DataFrame(robust_scaler.fit_transform(df), columns=df.columns)
print("\nRobust Scaled Dataset:")
print(df_robust)

```

Original Dataset:

	Age	Income	Monthly_Spending
0	25	50000	2000
1	30	60000	2500
2	35	70000	3000
3	40	80000	3500
4	45	90000	4000

Min-Max Scaled Dataset:

	Age	Income	Monthly_Spending
0	0.00	0.00	0.00
1	0.25	0.25	0.25
2	0.50	0.50	0.50
3	0.75	0.75	0.75
4	1.00	1.00	1.00

Standardized Dataset (Z-score):

	Age	Income	Monthly_Spending
0	-1.414214	-1.414214	-1.414214
1	-0.707107	-0.707107	-0.707107
2	0.000000	0.000000	0.000000
3	0.707107	0.707107	0.707107
4	1.414214	1.414214	1.414214

Robust Scaled Dataset:

	Age	Income	Monthly_Spending
0	-1.0	-1.0	-1.0
1	-0.5	-0.5	-0.5
2	0.0	0.0	0.0
3	0.5	0.5	0.5
4	1.0	1.0	1.0

LAB SESSION # 09

```
In [12]: # =====  
# Interactive Data Visualization Lab  
# =====  
  
import pandas as pd  
import plotly.express as px  
import plotly.io as pio  
  
# FIX for Plotly display error  
pio.renderers.default = "browser"  
  
# Create dataset  
data = {  
    "Country": ["Pakistan", "India", "China", "USA", "UK"],  
    "Population": [241, 1429, 1412, 339, 68],  
    "GDP": [0.37, 3.7, 18.3, 26.9, 3.3],  
    "Life Expectancy": [67.3, 69.7, 78.2, 77.5, 81.2]  
}  
  
df = pd.DataFrame(data)  
print(df)
```

```

# Bar Chart
px.bar(
    df,
    x="Country",
    y="Population",
    title="Population by Country"
).show()

# Line Chart
px.line(
    df,
    x="Country",
    y="GDP",
    title="GDP by Country",
    markers=True
).show()

# Scatter Plot
px.scatter(
    df,
    x="GDP",
    y="Life Expectancy",
    size="Population",
    color="Country",
    title="GDP vs Life Expectancy"
).show()

# Pie Chart (Line 35 FIXED)
px.pie(
    df,
    names="Country",
    values="Population",
    title="Population Distribution"
).show()

```

	Country	Population	GDP	Life Expectancy
0	Pakistan	241	0.37	67.3
1	India	1429	3.70	69.7
2	China	1412	18.30	78.2
3	USA	339	26.90	77.5
4	UK	68	3.30	81.2

LAB SESSION # 10

```

In [6]: import wdata
import pandas as pd
import matplotlib.pyplot as plt

countries = {"PAK": "Pakistan", "USA": "United States"}
indicators = {"SP.POP.TOTL": "Population"}
df = wdata.get_dataframe(indicators, country=countries)

df = df.reset_index()
df['date'] = pd.to_numeric(df['date'])

```

```

df = df.sort_values('date')

plt.figure(figsize=(10, 6))
for country in df['country'].unique():
    subset = df[df['country'] == country]
    plt.plot(subset['date'], subset['Population'], label=country)

plt.title("Population over Time")
plt.xlabel("Year")
plt.ylabel("Total Population")
plt.legend()
plt.grid(True)
plt.show()

```

